

Lab 1. Ticket enhancer

Saturday walkthrough. Claude Code plugin.

A vague GitHub issue in. A ready contract out.

25 minutes of typing. Fork setup is already done.

Work from `labs/lab1_enhancer`. Not the repo root.



What you will build

A Claude Code plugin that grooms every open draft ticket in **your** fork, one poll at a time.

PIECE	PATH YOU CREATE
Judge agent	<code>.claude/agents/enhancer-judge.md</code>
Doer agent	<code>.claude/agents/enhancer-doer.md</code>
Field check	<code>.claude/skills/enhancer-loop/scripts/check_fields.py</code>
Stop check	<code>.claude/skills/enhancer-loop/scripts/check_stop.py</code>
Orchestrator skill	<code>.claude/skills/enhancer-loop/SKILL.md</code>

Already in the folder: `Taskfile.yml`, `bin/*.sh`, `config.json.example`, `.claude/settings.json`, four prompt files.

Why this lab exists

A prompt that grooms a ticket once is a demo. A loop that polls GitHub, drafts, judges, and stops is a product.

- Nobody sits in chat driving it.
- The judge cannot edit the ticket it grades.
- Ready is arithmetic, not a model claim.
- `task poll-forever` is a seminar lie. Production is an event. See `GITHUB-ACTIONS.md`.

Final outcome. T900, T901, T902 exist as GitHub issues. One `task run` grooms the stubs. A human LGTM on a green rubric sets `ready` and `loop: implementer`.

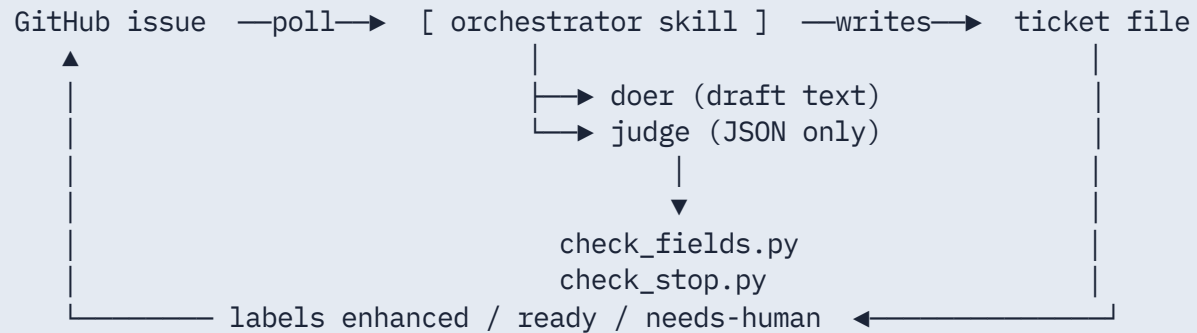
Learning objectives

After this lab you can:

- Configure a Claude Code plugin with two subagents and one skill
- Implement a judge that holds **no write tool**
- Implement a doer that returns a draft as text, not as a file write
- Validate ready with `check_fields.py`, not with the model's own JSON
- Stop on budget or a repeated signature with `check_stop.py`
- Troubleshoot a poll that labels `enhanced` without rewriting the ticket
- Point the same loop at GitHub Actions later, without changing the exits

Starting architecture

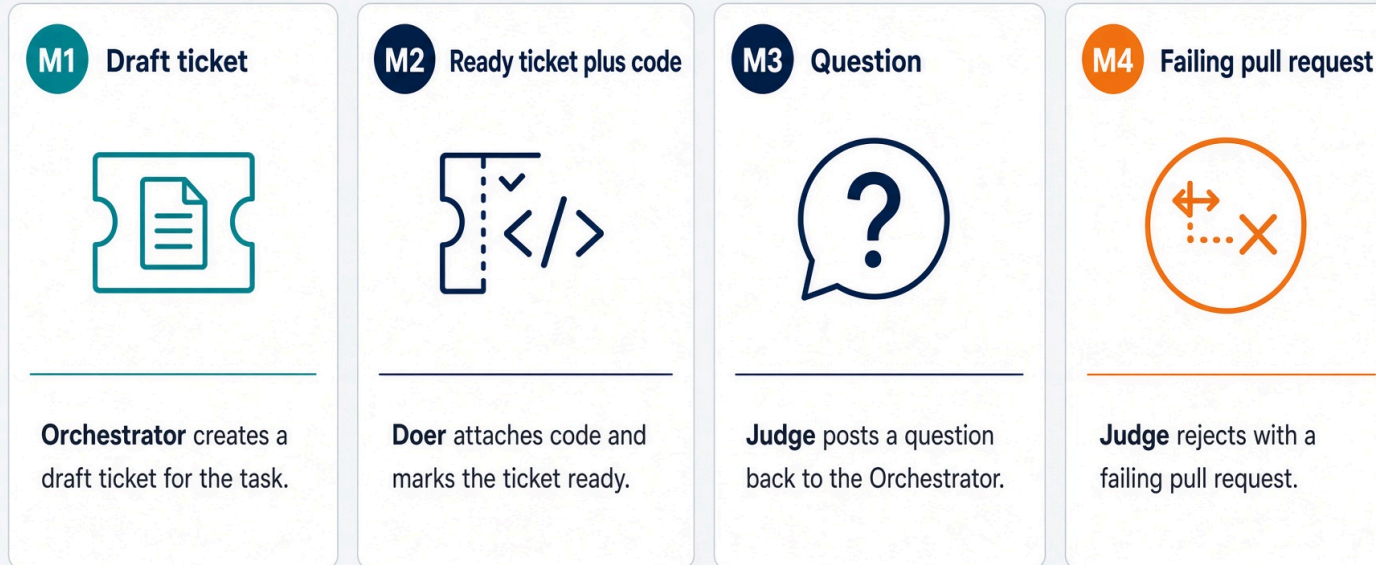
Components that already exist (gray). Components you create (navy).



Trigger lives **outside**: `task run`, `task poll-forever`, `/loop`, or **Actions**.
The skill runs **one step and exits**.

Four objects, this hour is tickets

Same graph: Orchestrator plus Doer plus Judge



Same graph every hour. Module 1 swaps in a draft ticket.

Orchestrator writes. Doer drafts. Judge scores. Human says LGTM.

Prerequisites

NEED

GitHub account and `gh`

Claude Code

Fork of northwind-field-crm, in **your** account`task`

Working directory

CHECK

`gh auth status``claude --version``never SpillwaveSolutions``task --version``cd labs/lab1_enhancer`

```
cp config.json.example config.json
# fill fork_owner with your GitHub username
task clone
```

`task clone` **writes** `../../work/northwind-field-crm`. One clone for every module.

Expected: a git checkout under `work/`, not an error about `fork_owner`.

Step 1. Start Claude in this folder

```
cd labs/lab1_enhancer
claude
```

Paste **one prompt at a time** from `prompts/claude-code.md`.

`.claude/settings.json` already denies:

```
Write/Edit ../../loops/**
Write/Edit ../../scripts/**
Write/Edit ../../work/**/tests/**
```

You are not editing the CRM tests. You are not rebuilding a shared engine.
There is no `loops/` package. Duplicate code on purpose.

Step 2. Judge agent

What. Create `.claude/agents/enhancer-judge.md`.

Why. A judge that can edit the ticket can grade itself.

```
---
name: enhancer-judge
description: Reads one ticket file and reports which required fields for its kind have real content. Never writes anything, never grades its own d
tools: Read, Grep, Glob
---
```

Entire final message is one JSON object:

```
{"kind": "bug", "present_fields": ["title", "steps"]}
```

List only fields with real content, not a bare heading.

Required fields, by kind

KIND	REQUIRED
bug	title (8+ characters), steps, expected, actual, environment
feature	problem, proposal, value, criteria (2+ a test can fail)
ui	feature, plus wireframe

Classify from title and body: crash / error / fails / regression → bug. form / page / button / screen / layout → ui. Otherwise feature.

The judge reports `present_fields`. It does **not** compute `missing_fields`. That is Python's job.

Step 3. Doer agent

What. Create `.claude/agents/enhancer-doer.md`.

Why. The draft must be judged before it can reach the real ticket.

Same tools: `Read`, `Grep`, `Glob`. **No write.**

Investigate `app/` in the target repo. Use the latest human GitHub comment as the strongest signal. Where neither settles a field, fill a stated guess. A missing field blocks the ticket. A stated guess does not.

The orchestrator writes `tickets/<id>.enhancer-candidate.md` from the doer's text. The doer does not write that file itself.

Step 4. Deterministic field check

What. `.claude/skills/enhancer-loop/scripts/check_fields.py`

The Judge agent decides which fields have real content (a model judgment). This script decides whether that adds up to ready (a fact).

```
python3 .claude/skills/enhancer-loop/scripts/check_fields.py --demo
```

Expected:

```
check_fields: all demo assertions passed
```

It reads `{"kind", "present_fields"}` and prints `missing_fields` and `ready`. It ignores any `missing_fields` the caller also sent. Invented field names are dropped.

Step 5. Deterministic stop

What. `.claude/skills/enhancer-loop/scripts/check_stop.py`

A stop condition trusted to a model's own judgment is a stop condition a model can talk itself past.

Input: `round, budget, signature, previous_signature`.

Output: `{"stop": bool, "reason": str or null}`.

Stop when the signature repeats (not on round 0), or when `round + 1 >= budget` (budget is 3).

```
python3 .claude/skills/enhancer-loop/scripts/check_stop.py --demo
```

Expected: `check_stop: all demo assertions passed.`

Step 6. Orchestrator skill. The eight rules

```
/enhancer-loop --repo <path> [--ticket <id>] [--simulate-comment "<text>"]
```

Hard rules from the answer's `SKILL.md`:

- A missing comment does not stop you. Do not fetch comments until `check_fields.py` says ready.
- The `enhanced` label is not the work. Adding it without rewriting the ticket file is a failed poll.
- Seed stubs are never ready.
- An issue opened in the GitHub UI is a ticket. Materialize a local file.
- `ready` comes from `check_fields.py`, never from the judge's own claim, never from a label, never from a comment other than exact LGTM.

Issue lookup. Order is load-bearing

1. State file `.harness/last-enhancer-<id>.json` field `github_issue`
2. Ticket frontmatter `github_issue`
3. Title search `--state all` for `[<id>]`

Never `--state open`. Never `gh issue create` from `task run`. Creating tickets is `task create-test-tickets`.

Closed issue → stop, say so. None found → tell them to run `task create-test-tickets`.

Strict improvement and the marker

Candidate `missing_fields` must be a **proper subset** of the current ticket. "Not worse" is not enough.

On keep: copy candidate over the real ticket, `gh issue edit --body (frontmatter stripped), --add-label enhanced`.

On no-op: leave the file and the issue body untouched.

Either way: delete the candidate. Post one comment containing:

```
<!-- enhancer-loop -->
```

When you later read comments, skip any body that contains that marker. **Do not filter by author.** The loop runs as your `gh` account. An author filter would drop your own LGTM.

Three exits. No fourth.

EXIT	WHEN	LABELS
pass	newest human comment is exactly LGTM and the rubric is already green	ready. Frontmatter state: ready, loop: implementer. Delete the state file.
escalate	same missing_fields twice	needs-human
escalate	round budget spent (3)	needs-human

LGTM on a red rubric finalizes nothing.

enhanced is history. It stays after ready.

State file. Memory for a memoryless job

<repo>/.harness/last-enhancer-<id>.json

```
{
  "github_issue": 42,
  "last_comment_id": 1000000001,
  "round": 1,
  "previous_signature": ["criteria", "value"]
}
```

A scheduled Actions job has no chat transcript. This file is how the next poll resumes. Deleted on pass. Frontmatter `github_issue` outlives it.

Step 7. Seed tickets and run one poll

```
task create-test-tickets && task run --
```

ID	KIND	TITLE
T900	bug	Search crashes on an empty query
T901	ui	Add a notes field to the customer page
T902	feature	Export tasks to CSV

```
task run is claude -p "/enhancer-loop --repo {{.TARGET}}".
```

Expected narration: one line per ticket, `waiting` / `escalated` / `passed`. First poll grooms stubs with no comment. Seed stubs are never ready.

Expected result after poll 1

- Ticket files under `work/northwind-field-crm/tickets/` have real fields, not a title plus one sentence
- Label `enhanced` on issues that were actually rewritten
- A marked comment on those issues
- `.harness/last-enhancer-T900.json` (and T901, T902) exist
- `check_fields.py` still reports missing fields on a stub that was not improved

Second `task run` with no new human comment: no-op on a ticket whose candidate was not better. Must **not** post a second identical reply.

Human LGTM, then pass

Comment `LGTM` on GitHub as yourself, on a ticket `check_fields.py` already calls ready.

Next poll:

- label `ready`
- frontmatter `state: ready` and `loop: implementer`
- state file deleted

That is the handoff into Lab 2.

For the seminar:

```
task poll-forever --
```

`while true: task run; sleep poll_interval.` It never self-stops. `Ctrl-C` when you are done. Use `poll_interval: 30s` in `config.json` while testing. `10m` is production.

Validation checklist

- ☐ `check_fields.py --demo` passes
- ☐ `check_stop.py --demo` passes
- ☐ Judge YAML lists `Read`, `Grep`, `Glob` only
- ☐ Doer YAML lists the same, no `Write`
- ☐ One `task run` rewrites at least one stub and labels `enhanced`
- ☐ Loop comments contain `<!-- enhancer-loop -->`
- ☐ A second poll does not duplicate that comment
- ☐ LGTM on a green rubric sets `ready`
- ☐ LGTM on a red rubric does not

Prompt 5 in `prompts/claude-code.md` diffs your `.claude/` against `../../solutions/sol1_enhancer/.claude/`.

Troubleshooting

SYMPTOM	LIKELY CAUSE	RESOLUTION
Skill not found	ran claude from the repo root	cd labs/lab1_enhancer
enhanced label, body unchanged	orchestrator labeled without rewriting	failed poll. Rewrite, then label.
Loop ignores your LGTM	filtered comments by author	filter on the marker, not the author
Infinite comments	marker missing on posted bodies	add <!-- enhancer-loop -->
gh issue create during task run	prompt drift	never create.task create-test-tickets only.
Closed issue reused	searched --state open only	lookup uses --state all
Hang on poll-forever	expected	ctrl-C. It never self-stops.

Fall behind

Nobody is graded. Save the attempt first.

```
# Lab 1 is the only lab with a drop-in
cp -R ../../solutions/sol1_enhancer/.claude .claude
```

See `FALL-BEHIND.md`.

Other tools, after class, not now:

TOOL	ANSWER FOLDER
Codex	solutions/sol1_enhancer_codex
OpenCode	solutions/sol1_enhancer_opencode
Grok Build	solutions/sol1_enhancer_grok_build
Agent SDK	solutions/sol1_enhancer_agent_sdk (issue 118 family)
Deep Agents	solutions/sol1_enhancer_deep_agents

Production. Actions, not a laptop loop

`SPEC.md` said the workflow file did not exist yet. It does now, as a copy-me.

- **Notes:** `labs/lab1_enhancer/GITHUB-ACTIONS.md`
 - **Workflow:** `labs/lab1_enhancer/workflows/enhance-on-issue.yml`
-

Copy the YAML onto **your CRM fork**. Listen to `issues` opened / edited / labeled and `issue_comment` created.

Skip `<!-- enhancer-loop -->`. Concurrency per issue. Label `agent-in-progress`. Cap `AGENT_MAX_ATTEMPTS`.

The trigger moves. The exits stay. A workflow that merges has left the lab.

Recap

What we built. A plugin: judge, doer, two Python checks, one orchestrator skill that polls once and exits.

What you learned. Scope is a missing method. Ready is arithmetic. The interesting run is the one that stops.

Takeaways

1. Trigger outside, exits inside.
2. Judge has no write tool.
3. `check_fields.py` computes ready. The model does not.
4. Marker, not author, when reading comments.
5. `task poll-forever` is a seminar stand-in. Actions is the deploy.

Closing line. The loop is the product. The prompt is not.

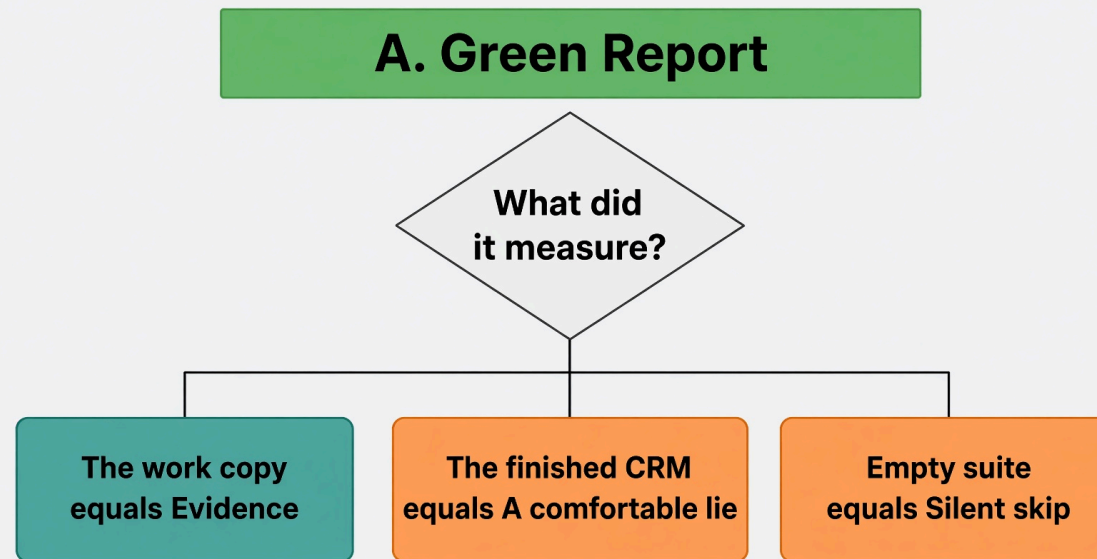
Lab 2. Ticket implementer and the harness

The centre of the workshop. 55 minutes. Never cut.

Already ticket in. A green rubric out.

25 minutes of typing. Fill `harness.py`. Nothing else.

Work from `labs/lab2_implementer`.



Why this lab exists

The loop you just built will lie to you. Edit forever. Declare victory on red. Stuff the window.

A true story from this repo: seven tests, green on every run, testing the wrong tree. The conftest put the finished answer on `sys.path` ahead of the work copy.

A check that reports success while measuring the wrong thing is worse than no check.

Artifact. A reusable evaluation harness: tests first, prove them red, code until green, judge, gate.

Learning objectives

- Implement `red_gate` so only **new** failing ids count
- Implement `score_attempt` as one call to `rubric.score`
- Implement `run_loop` so Python counts retries, not the model
- Configure two doers with disjoint write scope
- Validate `--doer none` escalates (honesty) and `--doer reference` can pass
- Troubleshoot the stall of computing rubric rows by hand

Starting architecture

Already in this folder: `contract.py`, `rubric.py`, `gates.py`, the stub, prompts.

You create: three function bodies in `harness.py`.

```
orchestrator (writes nothing, owns budget)
planner      writes steps.jsonl
test_implementer writes tests/**
code_implementer writes app/**, denied tests/**
judge        no write method

      |
      ▼
red gate → ten-row rubric → gates.decide
      |
      ▼
.harness/last-implementer.json
.harness/receipt.json
```

This is Graph Engineering: each acceptance criterion becomes two nodes in `steps.jsonl`, a test step and a code step. Derived, not generated.

There is no `loops/` package. Do not recreate it.

Prerequisites

```
cd labs/lab2_implementer
```

Pick one tool and paste its prompt:

```
claude -p "$(cat prompts/claude-code.md)"  
# or: codex exec "$(cat prompts/codex.md)"  
# or: grok -p "$(cat prompts/grok-build.md)"  
# or: opencode run "$(cat prompts/opencode.md)"
```

CRM clone from Lab 1 should already sit at `../..work/northwind-field-crm`.

`.claude/settings.json` **denies** writes to `loops/`, `scripts/`, and `work/**/tests/**`.

The stub. Three functions. The order is the lesson

```
def red_gate(before: RunResult, after: RunResult) -> set[str]:  
    raise NotImplementedError("fill me in")  
  
def score_attempt(contract: Contract, **evidence) -> rubric.Score:  
    raise NotImplementedError("fill me in")  
  
def run_loop(contract: Contract, budget: int = 3) -> dict:  
    raise NotImplementedError("fill me in")
```

Order, from the module docstring:

```
tests first -> prove them red -> code until green -> judge -> gate
```


`red_gate`. **New failing ids, not any failing ids**

A test that already existed and still fails is not proof of a new contract.

A test that passes before any code exists proves nothing.

An empty result must stop the loop.

Body Rick types (and the Deep Agents port ships):

```
def red_gate(before, after) -> set[str]:  
    seen = before.junit.passed_ids | before.junit.failed_ids  
    return {tid for tid in after.junit.failed_ids if tid not in seen}
```

The DA core names this `implementer._new_test_ids`.

score_attempt. **One line. Do not compute rows**

"The tests passed" is one row of ten. Absent kwargs become failing rows on purpose.

```
def score_attempt(contract: Contract, **evidence) -> rubric.Score:
    return rubric.score(contract=contract, **evidence)
```

That is the whole function. The common stall is writing ten `if` blocks.

#	ROW	EVIDENCE
1	tests_ran	junit exists and is non-empty
2	tests_passed	failures+errors == 0
3	red_first	red_ids non-empty
4	coverage_floor	line-rate vs floor (80)
5	criteria_covered	every criterion has a passing test
6	steps_done	no open plan steps
7	ui_has_e2e	UI files changed ⇒ e2e green
8	format_clean	format task ok
9	lint_clean	lint task ok
10	write_scope	no violations (and scope was checked)

`run_loop`. Python holds the retries

```
def run_loop(contract, budget: int = 3, ticket_id: str = "T001", doer: str = "reference") -> dict:
    return implementer.run(
        repo=contract.repo, ticket_id=ticket_id, budget=budget, doer=doer,
    )
```

Saturday this folder has no `implementer.py`. Rick types a body that calls `gates.decide` in a `while` loop, or you watch him fill it. The shipped eight-step loop lives in `solutions/sol2_implementer_deep_agents/implementer.py`.

Pass `doer` through. Hardcoding `reference` makes `--doer none` impossible.

Two doers. Scope is a missing path

From `contract.py` defaults:

```
planner          write_allow: steps.jsonl
test_implementer write_allow: tests/**
code_implementer write_allow: app/**, src/**  write_deny: tests/**
judge            write_allow: []             write_deny: **
```

Deny always beats allow. Empty allow permits nothing. Judge has no `write` method.

The code implementer cannot weaken a test, not because it was told not to, but because it holds no write path to one.

`gates.decide`. **Three exits, no fourth**

1. Rubric green (`judge_done` is `None`) → pass
2. Rubric green and final judge agrees → pass
3. Rubric green, judge says not done → escalate
4. `signature == previous_signature` → escalate (not converging)
5. money budget spent → escalate
6. iteration budget spent → escalate
7. else → retry, `final_attempt` if the next one is last

`signature` is the tuple of **failed row names**, not the wording. Two equal signatures mean the last attempt changed nothing.

Commands. Live first.

`task loop:implementer` is gone from the root Taskfile. `loops/` was deleted on purpose.

Saturday self-check:

```
task test    # python3 -c "import harness; print('ok')"
```

To actually run the eight-step loop, use the Deep Agents port (after class, or as the instructor demo):

```
cd ../../solutions/sol2_implementer_deep_agents
python3 harness.py --repo ../../work/northwind-field-crm --ticket T001 --doer reference
python3 harness.py --repo ../../work/northwind-field-crm --ticket T001 --doer none
```

The lab README verify is `task test`. Demo from the Deep Agents port.

Expected results

`--doer none` writes nothing. The red gate must escalate:

```
gate: escalate
reason: red gate: no new test was observed failing. A test that passes before any code exists proves nothing.
```

If this run were green, the harness would be lying.

`--doer reference` copies known-good into `tests/**` then `app/**`, each phase bound by that role's WriteScope. Expect ten PASS rows and `gate: pass`.

Receipt. Three claims or nothing

`scripts/receipt.py` writes `.harness/receipt.json`.

1. Suite passed (exit 0 **and** readable junit **and** no failures)
2. Ran against **this tree** (`tree_hash` of tracked plus untracked)
3. Ran **after** the newest source edit

A zero exit code with no test report is the silent-skip bug wearing a green shirt.

The push-gate refusal you hit live lives on the CRM clone as a Claude Code hook, not under `labs/lab2_implementer/.claude/`. Read the refusal. It is the lesson.

Troubleshooting

SYMPTOM	LIKELY CAUSE	RESOLUTION
Computing ten rows by hand	stall on <code>score_attempt</code>	<code>return rubric.score(contract=contract, **evidence)</code>
Returning all failed ids	<code>red_gate</code> used after <code>.failed</code>	subtract before ids
Edited loops/ or <code>rubric.py</code>	ignored the prompt	fill only <code>harness.py</code>
<code>task loop:implementer</code> missing	engine deleted	demo via <code>sol2_implementer_deep_agents/harness.py</code>
<code>--doer</code> none is green	red gate not stopping	empty new-ids must escalate
<code>import _root</code> fails	leftover TROUBLESHOOTING	lab stub does not import it

Fall behind

There is no drop-in `harness.py`. Watch Rick finish. Save first:

```
cp harness.py harness.py.my-attempt
```

See `FALL-BEHIND.md`. Restore with `git checkout -- harness.py`.

Take-home, not Saturday: issues 118. `labs/takehome/deep-agents/loop.py` and `labs/takehome/agent-sdk/loop.py` fill build and run. Those ports are a different runtime, not a drop-in for this stub.

Recap

What we built. Three functions that make a loop honest: new red ids, a ten-row rubric, Python-owned retries.

Takeaways

1. Two doers. Disjoint scope. The judge has no write method.
2. A test that passes before any code exists proves nothing.
3. "The tests passed" is one row of ten.
4. Same signature twice is stop.
5. A receipt proves three things, or it proves nothing.

Lab 3. Research assistant over MCP

A question in. A cited brief out. Same graph, different object.

25 minutes. Fill `loop.py`. Two functions. Nothing else.

Work from `labs/lab3_research`.

Same graph Orchestrator plus Doer plus Judge



What you will build

Two function bodies:

```
def plan_questions(question: str) -> list[str]: ...  
def check_brief(body: str, sources: list[str]) -> brief.BriefScore: ...
```

Already shipped: `brief.py` (the judge). Do not reimplement grounding.

Final outcome. Three checkable sub-questions. Four arithmetic rows on the brief. No model in the judge.

Why this lab exists

A code loop stops when tests go green. A research loop has no equivalent. "Keep searching until confident" is not a stop condition.

Tool output is untrusted input. Search results are a document the internet wrote, not a system prompt.

Saturday uses the fixture backend. No signup form.

Spillwave Solutions | spillwave.com

Learning objectives

- Implement `plan_questions` as three checkable sub-questions
- Implement `check_brief` as `return brief.check(body, sources)`
- Configure a tool boundary: search in, merge denied
- Validate `has_sources`, `grounded`, `cited`, `style`
- Troubleshoot the stall of reaching for a model judge

Spillwave Solutions | spillwave.com

Starting architecture

```
orchestrator  owns budget + exits. Writes nothing. Sees summaries.  
+-- researcher  search tools only. Isolated context. No write.  
+-- writer      brief.md + work/research/** only.  
+-- judge       brief.check. Citations are arithmetic.
```

Allowed: search, write this loop's own output folder.

Denied: merge, deploy, ticket state, CRM edits.

Prerequisites

```
cd labs/lab3_research
claude -p "$(cat prompts/claude-code.md)"
```

Worth reading: `brief.py`, `MCP.md`.

`.mcp.json` at the repo root ships `context7`. Perplexity is optional. Fixture when the room has no wifi.

`.claude/settings.json` denies writes to `loops/`, `scripts/`, and CRM tests. There is no `loops/` package.

Spillwave Solutions | spillwave.com

The stub

```
def plan_questions(question: str) -> list[str]:  
    raise NotImplementedError("fill me in")  
  
def check_brief(body: str, sources: list[str]) -> brief.BriefScore:  
    raise NotImplementedError("fill me in")
```

Fill only `loop.py`. Do not edit `brief.py`. Do not edit `solutions/`.

Spillwave Solutions | spillwave.com

`plan_questions`. **Three strings is enough**

A plan step you cannot check is a wish. The common stall is over-designing this function. This is Graph Engineering at lab scale: three checkable nodes, not a wish list.

```
def plan_questions(question: str) -> list[str]:  
    return [  
        question,  
        f"{question} common mistake",  
        f"{question} how to verify",  
    ]
```

That is the body in `solutions/sol3_research_deep_agents/researcher.py`. Swapping in a model planner is the stretch. Downstream checks do not change.

check_brief. **Arithmetic. No model**

```
def check_brief(body: str, sources: list[str]) -> brief.BriefScore:
    return brief.check(body, sources)
```

ROW		PASS WHEN
has_sources	bool(sources)	
grounded	every [n] is in 1..len(sources)	
cited	every claim paragraph has a [n]	
style	zero em dashes outside code spans	

A confident sentence nobody can trace is the failure that matters.

Why style is a row

House style forbids em dashes. A model will argue. Python will not.

`brief.strip_em_dashes` stashes code spans, then comma-heavy lines become `:`, light lines become `;`.

Spillwave Solutions | spillwave.com

Commands. Live first.

`task loop:research` is gone with `loops/`.

Saturday self-check:

```
task test    # import loop; print('ok')
```

Runnable filled loop, after class or as the instructor demo:

```
cd ../../solutions/sol3_research_deep_agents
python3 loop.py --question "sqlalchemy nullable datetime column" --backend fixture
```

The question is boring on purpose. It matches `fixtures/research.json`.

Expected fixture result

```
PASS  has_sources      3 sources retrieved
PASS  grounded         every citation resolves
PASS  cited            every paragraph cites a source
PASS  style            0 em dashes

backend: fixture      budget: $0.00 / $0.20 (soft $0.10), 3/8 calls
gate:   pass
```

Empty findings → no body claims → has_sources fails → escalate. Never ship an uncited brief.

Budget. Soft warns. Hard raises

Live: `max_usd=0.20`, `max_calls=8`, `soft_usd=0.10`. Perplexity costs 0.006 per call. Fixture costs nothing, which is why Saturday still teaches the cap.

The ninth search does not run. `BudgetExceeded` is a `RuntimeError`, not a nudge.

Four stops: call budget, dollar budget, stable failure, no-source escalates.

Spillwave Solutions | spillwave.com

Troubleshooting

SYMPTOM	CAUSE	FIX
Reaching for a model in <code>check_brief</code>	stall	<code>return brief.check(...)</code>
Over-designed planner	stall	three strings
<code>task loop:research</code> missing	engine deleted	<code>sol3_research_deep_agents/loop.py</code>
Uncited brief shipped	skipped <code>has_sources</code>	empty findings escalate
Em dashes in the brief	model argued	<code>brief.strip_em_dashes</code>

Fall behind

```
cp loop.py loop.py.my-attempt
```

Watch Rick type. There is no drop-in. Later: `git checkout -- loop.py`.

Take-home ports are a different runtime, not a drop-in for this stub.

Spillwave Solutions | spillwave.com

Recap

Same graph as Module 1. New object: a question. New wall: one search tool.

Cost is an architecture problem, not a pricing problem. Do not shop for a cheaper model first.

Takeaways

1. Three sub-questions is a plan you can check.
2. Citations are arithmetic.
3. The orchestrator sees a summary, never the dump.
4. Merge is not a tool this loop holds.
5. Saturday object is a cited brief. Paper pipelines are take-home.

Lab 4. Broken PR fixer, unattended

A failing branch in. A green one out, or an honest explanation of why not.

18 minutes, not 25. Energy is lowest. Keep moving.

Work from `labs/lab4_fixer`.

What you will build

Two functions in `loop.py`:

```
def summarize_failure(run_result: RunResult) -> str: ...  
def repair_until_green(contract: Contract, budget: int = 3) -> dict: ...
```

Already shipped: `contract.py` (the repo contract, 344 lines).

Final outcome. A loop that can give up out loud.

Why this lab exists

Nobody is watching. Exits matter more than successes.

Giving up is allowed. Giving up silently is the bug.

A model that may both act and verify can invent its own evidence. That is why the receipt exists.

Spillwave Solutions | spillwave.com

Learning objectives

- Implement `summarize_failure` so the error is not lost in the middle of a log
- Implement `repair_until_green` with four stop paths
- Stash Module 2 work **before** `broken-pr` checkout
- Validate `--doer none` leaves "A human should take this one"
- Troubleshoot a loop that wants to delete an earlier lab's work

Starting architecture

```
Trigger → fixer.run
        judge: contract.run("test") → junit
        code_implementer: app/**, denied tests/**
        optional research once (2 calls, $0.05)
→ .harness/last-fixer.json
→ Human merge. Loop never gets a merge tool.
```

Cast is three roles, not five. No planner. The work is defined by what is red.

Stash first. Say it out loud

```
git -C ../../work/northwind-field-crm stash --include-untracked
```

`checkout()` on a dirty tree refuses rather than deleting Lab 2 work:

```
cannot switch to broken-pr: northwind-field-crm still holds
the work from an earlier lab.
  keep it:      git -C ... stash --include-untracked
  discard it:   git -C ... checkout -- . && git -C ... clean -fd
```

That refusal is on brand.

CRM branches

BRANCH	STATE
main	no due dates, ~75% coverage, below floor
known-good	due dates implemented, everything green
broken-pr	dropped the null guard, one test red

--branch broken-pr is what makes this real. Point it at a green branch and it reports a pass and proves nothing. Same shape as the red gate.

The stub

```
def summarize_failure(run_result: RunResult) -> str:  
    raise NotImplementedError("fill me in")  
  
def repair_until_green(contract: Contract, budget: int = 3) -> dict:  
    raise NotImplementedError("fill me in")
```

Fill only `loop.py`.

Spillwave Solutions | spillwave.com

`summarize_failure`

Sending the whole log puts the failure in the middle. Lost in the Middle, third time today.

```
def summarize_failure(run_result) -> str:
    failed = sorted(run_result.junit.failed_ids)
    lines = [f"{len(failed)} failing: {' '.join(failed[:5])}" if failed else ["the suite is red"]]
    error = ERROR_IN_OUTPUT.search(run_result.output or "")
    if error:
        lines.append(error.group(0).strip()[:200])
    return "\n".join(lines)
```

Four stop paths

1. Suite green → pass
2. Suite never ran → escalate on round 1
3. Same failing ids twice → escalate, leave a comment
4. Budget spent → escalate, "A human should take this one"

Scope violations escalate even if the suite is green. Reaching green by editing the failing test is not a fix.

Commands. Live first.

`task loop:fixer` is gone with `loops/`.

Saturday: `task test imports loop`.

Runnable filled loop:

```
cd ../../solutions/sol4_fixer_agent_sdk
python3 loop.py --repo ../../work/northwind-field-crm --branch broken-pr --doer reference
python3 loop.py --repo ../../work/northwind-field-crm --branch broken-pr --doer none
```

Expected `--doer none`

```
attempt 1: 1 failing -> retry
  1 failing: tests.test_overdue::test_overdue_ignores_tasks_with_no_due_date
attempt 2: 1 failing -> escalate

gate: escalate
The fixer gave up.
A human should take this one.
```

`--doer` reference **against** `broken-pr`: `gate: pass`, **files copied from** `known-good` **inside** `app/**` **only**.

MAST, in one slide

Most agent failures are not model failures. 1,642 traces. Cemri et al. arXiv:2503.13657.

CATEGORY	SHARE	THIS HOUR
System design	41.8%	graph, scope, budget, trigger
Inter-agent	36.9%	summaries, not dumps
Verification	21.3%	pytest, receipt

Every one of those three is something you build, not something you buy.

Troubleshooting

SYMPTOM	CAUSE	FIX
Checkout refuses	Lab 2 files dirty	stash, out loud
--doer none is green	loop is lying	same ids twice escalate
Edited tests/**	scope not enforced	deny list on the coder
Silent give-up	missing comment	"A human should take this one."

Recap

Takeaways

1. Stash first.
2. Giving up is allowed. Giving up silently is the bug.
3. Merge is never a tool.
4. If you cannot read the last score, you cannot debug at 2 a.m.

The loop is the product. The prompt is not.

Spillwave Solutions | spillwave.com

sol1_enhancer_codex

Same loop as Lab 1. Isolation is a **process sandbox**, not a role flag.

Codex has no per-agent tool list. `$enhancer-judge` inside the orchestrator session inherits `workspace-write`. The jail is `bin/role.sh` starting a child `codex exec -s read-only`.

Work from `solutions/sol1_enhancer_codex`.

What you will build

A Codex skill pack that grooms draft tickets the same way Saturday's Claude plugin does.

PIECE	PATH
Orchestrator	<code>.agents/skills/enhancer-loop/SKILL.md</code>
Judge skill	<code>.agents/skills/enhancer-judge/SKILL.md</code>
Doer skill	<code>.agents/skills/enhancer-doer/SKILL.md</code>
Field check	<code>.agents/skills/enhancer-loop/scripts/check_fields.py</code>
Stop check	<code>.agents/skills/enhancer-loop/scripts/check_stop.py</code>
Jail	<code>bin/role.sh</code>
Fence probe	<code>bin/fence_check.sh</code>

`check_fields.py` and `check_stop.py` are byte-identical to the lab.

Why Codex is a different lesson

Claude isolates by omitting Write from YAML. Codex cannot. Every in-session skill inherits the parent's sandbox.

If the orchestrator runs under `workspace-write`, an in-session `$enhancer-judge` can edit the ticket it grades.

The product is still: vague issue in, ready contract out. The new knob is a child process.

Learning objectives

- Configure Codex skills under `.agents/skills/`
- Implement `bin/role.sh` so judge and doer cannot write
- Troubleshoot the four Codex traps
- Validate with `task fence-check` then `task run`
- Deploy the same loop on issue events (`ENHANCER_BACKEND=codex`)

Spillwave Solutions | spillwave.com

Starting architecture

```
task run
├─ codex exec -s workspace-write (orchestrator)
│   └─ bin/role.sh enhancer-judge
│       └─ codex exec -s read-only
├─ bin/role.sh enhancer-doer
│   └─ codex exec -s read-only
```

Trigger still lives outside. One poll, then exit. Labels, marker, and three exits are unchanged.

Prerequisites

```
cd solutions/sol1_enhancer_codex
codex --version
cp config.json.example config.json  # fork_owner
task clone
task fence-check
```

`task fence-check` fails if `~/.codex/config.toml` has `trust_level = "trusted"`. That setting silently turns the orchestrator fence off. Judge and doer stay read-only either way.

The fence. Flags live in the script

```
# bin/role.sh. The flags are not in SKILL.md on purpose.
codex exec -s read-only --cd "$DIR" -o "$OUT" "\$$SKILL $*" \
  </dev/null >/dev/null 2>&1
cat "$OUT"
```

< /dev/null is load-bearing. `codex exec` appends open stdin and waits for EOF. `task` pipes stdin. Without the redirect the child hangs with no error.

`--cd` lands in this folder so Codex finds `.agents/skills`. Ticket paths are passed absolute. A read-only process may still read outside its workspace.

task run

```
codex exec
-s workspace-write
-c sandbox_workspace_write.network_access=true
--add-dir {{.TARGET}}
--add-dir {{.CODEX_HOME_DIR}}
"\$enhancer-loop --repo {{.TARGET}} {{.CLI_ARGS}}"
< /dev/null
```

Escape `\$enhancer-loop` or the shell eats `$enhancer`.

`--add-dir $HOME/.codex` is required. A child `codex exec` cannot start without it.

AGENTS.md split by role

A first draft said "run judge through `bin/role.sh`". The judge read it, obeyed, started another judge. Infinite recursion.

Now:

- Judge and doer ignore orchestrator instructions
- Orchestrator never invokes `$enhancer-judge` as an in-session skill
- It shells out through `bin/role.sh`

Read `IMPLEMENTATION_NOTES.md` before you "simplify" this.

Three model processes per round

1. Judge scores the current ticket
2. Doer drafts a replacement body
3. Judge scores the candidate

Budget five minutes per poll. `timeout 420 task run --` while developing.

No self-reinvoke. `codex exec` exits when the turn ends. `task poll-forever` is still the seminar scheduler.

Commands and expected result

```
python3 .agents/skills/enhancer-loop/scripts/check_fields.py --demo
python3 .agents/skills/enhancer-loop/scripts/check_stop.py --demo
task create-test-tickets
timeout 420 task run --
```

Expected: T900/T901/T902 grooms, `enhanced` label after a real rewrite, comment with `<!-- enhancer-loop -->`, state file under `.harness/`.

Second poll with no new human comment: no-op. Must not post again.

Troubleshooting

SYMPTOM	CAUSE	FIX
Silent hang	open stdin	<code>< /dev/null</code> on every <code>codex exec</code>
Nested judges	AGENTS.md recursion	split instructions by role
Orchestrator can write as judge	<code>trust_level = trusted</code>	<code>task fence-check</code>
Child cannot start	missing <code>--add-dir ~/.codex</code>	add it on <code>task run</code>
<code>\$enhancer</code> empty	unescaped \$ in Taskfile	<code>\\$enhancer-loop</code>

Validation checklist

- ☐ `task fence-check` is green
- ☐ `--demo` scripts pass
- ☐ Judge process is `-s read-only`
- ☐ One poll rewrites a stub and labels `enhanced`
- ☐ Marker is on the posted comment
- ☐ LGTM plus green rubric sets `ready`

GitHub Actions

Set `ENHANCER_BACKEND=codex` in the copy-me workflow:

```
labs/lab1_enhancer/workflows/enhance-on-issue.yml
```

Hosted runners must have `codex` installed. If they do not, use Agent SDK or Claude Code on Actions. Keep Codex on a laptop.

The trigger starts one poll. This folder still owns the exits. Skip `<!-- enhancer-loop -->`.

Spillwave Solutions | spillwave.com

Recap

Isolation in Codex is a process sandbox, not a role flag.

The loop did not change. Ready is still `check_fields.py`. The judge still has no write path.

Takeaways

1. Flags live in `bin/role.sh`, not in the skill.
2. Close stdin.
3. Split AGENTS.md by role or you recurse.
4. Same three exits as Saturday.

sol1_enhancer_opencode

Same loop. Jail is **permission deny**, not a child process.

Native `.opencode/` discovery. Not a copy of `.claude/`. Not an OpenCode JS plugin. No `plugin.json`.

Spillwave Solutions | spillwave.com

What you will build

PIECE	PATH
Judge	<code>.opencode/agents/enhancer-judge.md</code>
Doer	<code>.opencode/agents/enhancer-doer.md</code>
Skill	<code>.opencode/skills/enhancer-loop/SKILL.md</code>
Scripts	<code>.opencode/skills/enhancer-loop/scripts/check_{fields,stop}.py</code>
Command	<code>.opencode/command/enhancer-loop.md</code>
Permissions	<code>opencode.json</code>

Saturday FALL-BEHIND.md still says "No OpenCode answer exists yet." That paragraph is stale. This folder is the answer.

Why OpenCode is a different lesson

Claude omits Write from a YAML tool list. Codex starts a second process. OpenCode puts `permission: edit: deny` on the subagent.

`--auto` does not override explicit deny. That is the claim to prove.

Spillwave Solutions | spillwave.com

Learning objectives

- Configure OpenCode agents with `mode: subagent`
- Deny edit, bash, and task on the judge
- Invoke headless via `--command enhancer-loop`
- Probe the jail with `cksum` before a live poll
- Point Actions at `ENHANCER_BACKEND=opencode`

Spillwave Solutions | spillwave.com

Starting architecture

```
opencode run --auto --command enhancer-loop
└─ agent: build (orchestrator, default)
    Task tool
        └─ enhancer-judge    permission.edit: deny
        └─ enhancer-doer     permission.edit: deny
```

--agent enhancer-judge fails: "is a subagent, not a primary agent." That is correct. The orchestrator is build.

Judge frontmatter. This is the type system

```
---  
description: Grade one enhancer ticket. Return JSON only.  
mode: subagent  
permission:  
  edit: deny  
  bash: deny  
  task: deny  
---
```

The body then says: if asked to write or edit, refuse and still return the JSON grade of the file as it currently is.

`opencode.json` and the command file

```
{ "permission": { "external_directory": "allow" } }
```

Without that, writes to `../../work/northwind-field-crm` become asks. Headless `--auto` would then stall.

`.opencode/command/enhancer-loop.md` pins `agent: build` and substitutes `$ARGUMENTS`.

Headless argv

```
opencode run
  --dir {{.TASKFILE_DIR}}
  --auto
  --command enhancer-loop
  --
  --repo {{.TARGET}} {{.CLI_ARGS}}
  < /dev/null
```

No built-in loop skill. No self-reinvoke. First poll is about six minutes. Cap it:

```
timeout 360 task run --
```

Jail probe before a live poll

Spawn the judge. Ask it to edit a file. `cksum` of that file must be identical.

If the file changed, `--auto` overrode deny and the port is wrong. Stop. Do not run `task create-test-tickets` yet.

Commands

```
cd solutions/sol1_enhancer_opencode  
cp config.json.example config.json  
task clone  
python3 .opencode/skills/enhancer-loop/scripts/check_fields.py --demo  
python3 .opencode/skills/enhancer-loop/scripts/check_stop.py --demo  
task create-test-tickets  
timeout 360 task run --
```

Expected result

Same as Saturday: stubs become real fields, `enhanced` after a rewrite, marked comment, state file.

`check_fields.py` still computes ready. The permission block did not have to change the rubric.

Spillwave Solutions | spillwave.com

Troubleshooting

SYMPTOM	CAUSE	FIX
"is a subagent, not a primary"	--agent enhancer-judge	orchestrator is build
Ask-loop on CRM writes	missing external_directory	set allow in opencode.json
Judge edited the ticket	deny not applied	probe with cksum
Six-minute hang	first poll is slow	timeout 360
FALL-BEHIND says no answer	stale doc	this folder is the answer

Validation

- [] Judge YAML has `edit: deny`
- [] Jail probe: file checksum unchanged
- [] `--demo` scripts pass
- [] One poll labels `enhanced` after a rewrite
- [] Marker on the comment

Spillwave Solutions | spillwave.com

GitHub Actions

`ENHANCER_BACKEND=opencode`. The job needs `opencode` on the runner.

Copy `labs/lab1_enhancer/workflows/enhance-on-issue.yml` onto **your** CRM fork. Do not enable it on the instructor repo.

Skip comments that contain `<!-- enhancer-loop -->`.

Recap

OpenCode isolation is a permission block on the subagent.

Same rubric. Same marker. Same exits. The loop did not have to change to make it run.

Spillwave Solutions | spillwave.com

sol1_enhancer_grok_build

Same loop. Artifact is a **project plugin** plus three registration shims.

Grok 1.0.5 does not register skills and agents from a project plugin on its own. Symlinks are the runnable surface.

What you will build

```
.grok/plugins/ticket-enhancer/plugin.json
.grok/plugins/ticket-enhancer/agents/{enhancer-judge,enhancer-doer}.md
.grok/plugins/ticket-enhancer/skills/enhancer-loop/...

.grok/skills/enhancer-loop      -> ../plugins/ticket-enhancer/skills/enhancer-loop
.grok/agents/enhancer-judge.md  -> ../plugins/ticket-enhancer/agents/enhancer-judge.md
.grok/agents/enhancer-doer.md   -> ../plugins/ticket-enhancer/agents/enhancer-doer.md
```

Do not `grok plugin install`. That copies into `~/.grok`. Edits here then stop taking effect.

Why Grok is a different lesson

Claude discovers `.claude/` in the folder you launched from. Grok 1.0.5 discovers project plugins only after trust, and only through the shim paths.

Until trusted, `task run` finds no skill and does nothing. Headless `grok -p` never prompts. Trust is step zero.

Spillwave Solutions | spillwave.com

Learning objectives

- Build a Grok project plugin (`plugin.json` name `ticket-enhancer`)
- Register it with three `ln -sf` shims
- Trust the **git root**, not this folder
- Deny MCP writes on a read-only judge
- Keep Grok off hosted Actions

Spillwave Solutions | spillwave.com

Starting architecture

```
grok -p "/enhancer-loop --repo ..."  
└─ skill via .grok/skills/enhancer-loop (symlink)  
    spawn_subagent  
        └─ enhancer-judge allowlist + disallowedTools  
        └─ enhancer-doer  allowlist + disallowedTools
```

Subagent tool on 1.0.5 is `spawn_subagent`. A README that says `task` is wrong on this CLI.

plugin.json

```
{  
  "name": "ticket-enhancer",  
  "description": "One poll-and-act step that grooms draft tickets.",  
  "version": "0.1.0"  
}
```

`--plugin-dir` does not exist on this CLI. The shims are the install.

Trust is step zero

Trust attaches to the **git root**, not `solutions/sol1_enhancer_grok_build`.

```
task trust
grok                # accept the prompt, maybe /plugins trust
grok inspect | grep -E "enhancer-loop|enhancer-judge|enhancer-doer"
```

Check **names**, **never counts**. The Plugins line counts directories (`1 agents` even with two files, none loaded).

Allowlist plus MCP deny

```
tools:  
  - read_file  
  - grep  
  - list_dir  
disallowedTools:  
  - search_tool  
  - use_tool
```

Allowlist blocks write and spawn. Grok still injects MCP tools. Without the deny, a "read-only" judge can write an issue via GitHub MCP.

task run

```
grok --always-approve
--deny "Edit({{.LAB_ROOT}}/scripts/**)"
--deny "Edit({{.LAB_ROOT}}/work/**/tests/**)"
-p "/enhancer-loop --repo {{.TARGET}} {{.CLI_ARGS}}"
```

-p must come last.

Hooks register but never fire on 1.0.5. An empty `debug.log` does not mean the skill failed to start.

Live progress: `grok --always-approve --output-format streaming-json -p "...".`

Commands

```
cd solutions/sol1_enhancer_grok_build
cp config.json.example config.json
task clone
task trust
python3 .grok/plugins/ticket-enhancer/skills/enhancer-loop/scripts/check_fields.py --demo
task create-test-tickets
task run --
```

Expected result

Same labels and state files as Lab 1. Same marker. Same three exits.

No Grok loop skill. Polling is always external: `task poll-forever` or Actions (poor fit, see next slide).

Troubleshooting

SYMPTOM	CAUSE	FIX
Skill not found	untrusted git root	<code>task trust</code> , then inspect names
Judge wrote an issue	MCP tools injected	<code>disallowedTools: search_tool, use_tool</code>
Edits have no effect	<code>grok plugin install</code> copied away	use the shims, never install
Empty debug.log	hooks never fire on 1.0.5	stream JSON instead

GitHub Actions

Hosted Actions is a poor fit (trust prompt, local shims). Set `ENHANCER_BACKEND` to `claude`, `agent-sdk`, or `deep-agents` on the runner.

Keep Grok on a laptop or a Droplet (`labs/extra-credit/ext_5_digitalocean`).

The exits do not change if you do run it under Actions. See `labs/lab1_enhancer/GITHUB-ACTIONS.md`.

Recap

Plugin on disk. Shims to register. Trust the git root. Deny MCP writes.

Same `check_fields.py`. Same three exits. The loop is the product. The runtime is not.

Spillwave Solutions | spillwave.com

sol1_enhancer_agent_sdk

Take-home. Python owns the loop. The model only drafts and grades.

Stop conditions cannot be talked past.

Needs `claude-agent-sdk` and an API key for a live poll. `task test` does not.

What you will build

FILE	JOB
enhancer.py	one poll-and-act step in Python
loop.py	CLI --once, --table-only
roles.py	ClaudeAgentOptions plus scope_hook
plugin/	same agents as Saturday, loaded with plugins=
check_fields.py	ready is arithmetic
check_stop.py	done, cost, max turns
tests/	pytest, stubs the SDK

The point is not that it runs. The point is that the rubric, the write scope, and the exits did not have to change.

Why Python holds the loop

A skill is a model following steps. A model can skip a step.

Here the parent session may only use the `Agent` tool. Python writes the candidate from the doer's text. Python calls `check_fields.py`. Python calls `check_stop.py`.

Learning objectives

- Configure `tools=[...]` so judge and doer hold no Write
- Implement `scope_hook` with the full deny envelope
- Validate `--table-only` prints judge writes = no
- Troubleshoot a hook that fails **open** because of a typo
- Name the marker gap vs Deep Agents
- Deploy `loop.py --once` on issue events

Spillwave Solutions | spillwave.com

Starting architecture

```
python loop.py --once --repo TARGET --ticket T900
└─ enhancer.py
    Agent tool
    │ └─ enhancer-judge  Read, Grep, Glob  output_format JSON
    └─ enhancer-doer    Read, Grep, Glob  text draft
Python writes tickets/<id>.enhancer-candidate.md
check_fields.py / check_stop.py
gh labels + .harness/last-enhancer-<id>.json
```

Scope. Two places, both required

1. `tools=[...].NO_WRITE` strips Edit, Write, Bash.
2. `PreToolUse` `hook` `roles.scope_hook`.

Empty `{}` = allow. Deny must be:

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "...",
  }
}
```

A typo fails **open**. Tests assert this shape key by key.

Parent session

```
allowed_tools=["Agent"]
disallowed_tools=NO_WRITE
CLAUDE_AGENT_SDK_DISABLE_BUILTIN_AGENTS=1
maxTurns=12
background=False
```

System prompt: "Python already owns the loop. Do not invoke the enhancer-loop skill."

Judge uses structured `output_format(JUDGE_SCHEMA)`. The hook still fires if a Write leaks.

Cast. `roleplan.py`

```
enhancer: orchestrator, doer, judge
orchestrator  tools: Task                writes: no
doer          tools: Read,Glob,Grep>Edit,Write,Bash  allow: tickets/**
judge         tools: Read,Glob,Grep,Bash             writes: no
```

Doer tools include Write in the table. Python still writes the candidate. The hook is the fail-closed belt.

Extra exits. Marker gap

`check_stop.py` also fires on `usd >= max_usd` and `turns >= max_turns`.

Known gap vs the plugin and Deep Agents: `Gh.latest_comment` does not filter `<!-- enhancer-loop -->`, and `comment()` does not append it. Do not copy that gap into a new port.

Spillwave Solutions | spillwave.com

Commands

```
cd solutions/sol1_enhancer_agent_sdk
pip install -r ../../requirements-takehome.txt
cp config.json.example config.json
task table          # python loop.py --table-only
task test           # pytest, no key, no clone
task checks
task clone && task create-test-tickets
task run            # python loop.py --once --repo TARGET
```

Outcome line: T900 waiting round 1, still missing value, criteria

Expected `--table-only`

role	writes	scope
orchestrator	no	nothing
doer	yes	tickets/**
judge	no	nothing

If judge prints `yes`, stop. The port is wrong.

Troubleshooting

SYMPTOM	CAUSE	FIX
Judge writes yes	tools list leaked Write	strip with NO_WRITE
Writes to app/	deny envelope typo	tests assert hookSpecificOutput
Duplicate comments	marker gap	copy Deep Agents filter
SDK import error in pytest	imported at module top	lazy import, tests stub

Spillwave Solutions | spillwave.com

Validation

- [] task test green without a key
- [] table: judge writes no
- [] deny shape matches tests key by key
- [] live poll grooms T900
- [] you know the marker gap exists

Spillwave Solutions | spillwave.com

GitHub Actions

ENHANCER_BACKEND=agent-sdk

```
python3 loop.py --once --repo "$GITHUB_WORKSPACE" --ticket "$TICKET"
```

Secret ANTHROPIC_API_KEY. **Copy** labs/lab1_enhancer/workflows/enhance-on-issue.yml **onto your CRM fork.**

Spillwave Solutions | spillwave.com

Recap

The harness is the product. The framework is not.

If a port imports a shared engine, the design leaked. This folder copies `roleplan.py` and `write_scope.py` on purpose.

Spillwave Solutions | spillwave.com

sol1_enhancer_deep_agents

Take-home. LangChain Deep Agents >= 0.7.

Python owns the loop. The doer **holds a write tool**, scoped to `tickets/**`. The judge still has none.

Model: `anthropic:claude-sonnet-5`.

What you will build

FILE	JOB
enhancer.py	poll-and-act in Python
roles.py	create_deep_agent, scoped write tool
skills/doer/SKILL.md	doer prompt
skills/judge/SKILL.md	judge prompt
check_stop.py	done, cost, max turns. Signature is not an exit
tests/	pytest, no SDK

Turn off the default `general-purpose` subagent. It ships harness FS tools. Leaving it on is how a "scoped" agent writes `app/`.

Learning objectives

- Hand each subagent its own tool list
- Put the path check **inside** the write tool
- Refuse with a sentence, not an exception
- Hide orchestrator write tools
- Compare `check_stop.py` to the plugin (signature is not an exit here)

Spillwave Solutions | spillwave.com

Starting architecture

```
python3 loop.py --once --repo TARGET
└─ create_deep_agent
    orchestrator: no write_file, edit_file, delete, execute
    doer: write tool, tickets/** only
    judge: read_file only
    FilesystemBackend(root_dir=crm, virtual_mode=True)
```

Scope. Three ways, all used

1. Per-subagent tool list. Judge is never handed write.
2. Path check inside the doer's write tool.
3. Harness fence: virtual FS root, permissions deny on orchestrator, allow `tickets/**` on doer, deny on judge.

```
ORCHESTRATOR_EXCLUDED_TOOLS = {write_file, edit_file, delete, execute}
```

Refusal is a sentence

```
@tool(f"write_{role.name}")
def write(path: str, content: str) -> str:
    try:
        scope.check(path)
    except ScopeViolation:
        return f"REFUSED. {role.name} may write {allowed}. {path} is outside that scope."
    (repo / path).write_text(content, encoding="utf-8")
    return f"wrote {path}"
```

An unformatted exception in an agent's context tends to start a retry loop. A short sentence tends to change the next action.

`check_stop.py` is a different contract

```
def check(*, done, turns, max_turns, spent_usd=0.0, max_usd=2.0) -> dict:
    if done: return {stop: True, reason: "done"}
    if spent_usd >= max_usd: return {stop: True, reason: "cost"}
    if turns + 1 >= max_turns: return {stop: True, reason: "max turns"}
    return {stop: False, reason: None}
```

Repeated signature is **not** an exit. Stuck work burns turns or dollars until cost or max-turns fires. `MAX_USD = 2.0`. Done waits for human LGTM.

Marker is implemented

`MARKER = "<!-- enhancer-loop -->"`. Fetch `per_page=100` because the default 30 plus a marker filter wedges long threads.

`already_acted_on` compares GitHub ids as **ints**. `"1000000001" <= "999999999"` is True as text and would skip a real new comment.

Spillwave Solutions | spillwave.com

Commands

```
cd solutions/sol1_enhancer_deep_agents
python3 loop.py --table-only
task test && task table && task checks
task clone && task create-test-tickets
task run
```

Taskfile dotenv: ['../.env', '.env']. First file that defines a var wins.

Expected table

role	writes	scope
orchestrator	no	nothing
doer	yes	tickets/**
judge	no	nothing

If judge prints `yes`, stop.

Troubleshooting

SYMPTOM	CAUSE	FIX
Writes app/	default general-purpose still on	turn it off
Retry storm	write tool raises	return a sentence
Skipped a new comment	id compared as text	compare as ints
Stopped on same gaps	copied plugin check_stop	signature is not an exit here

Validation

- `[] table: judge no`
- `[] pytest green without a key`
- `[] doer write to tickets/T900.md works`
- `[] doer write to app/main.py returns REFUSED`
- `[] marker on posted comments`

GitHub Actions

ENHANCER_BACKEND=deep-agents. **Secret** ANTHROPIC_API_KEY.

```
python3 loop.py --once --repo "$GITHUB_WORKSPACE" --ticket "$TICKET"
```

Same copy-me workflow as the other sol ports.

Spillwave Solutions | spillwave.com

Recap

Deep Agents is a harness. The subagent tools list replaces the parent. Python still owns the outer budget.

Three belts: missing tool, in-tool path check, harness profile. All three.

Spillwave Solutions | spillwave.com

sol2_implementer_deep_agents

The working filled Lab 2 loop. Take-home issue 118.

Python holds Pass / Retry / Escalate. Deep Agents is the maker. The red gate is `junit.xml`.

Spillwave Solutions | spillwave.com

What this folder is

FILE	ROLE
<code>harness.py</code>	<code>red_gate</code> , <code>run_loop</code> , CLI
<code>implementer.py</code>	eight-step loop
<code>doers.py</code>	<code>none</code> / <code>reference</code> / <code>cli</code> / Deep Agents
<code>rubric.py</code> / <code>gates.py</code> / <code>contract.py</code>	copies, not a library
<code>roles.py</code>	<code>create_deep_agent</code>
<code>tests/</code>	judge read-only, refuse <code>tests/**</code>

Why it exists

Saturday fills three stubs. There is no drop-in `harness.py`. This folder is the shipped eight-step answer.

`task loop:implementer` is gone from the root Taskfile. Demo from here. Each criterion becomes two nodes in `steps.jsonl`.

Spillwave Solutions | spillwave.com

Learning objectives

- Walk `implementer.run` step by step
- Prove `--doer none` escalates
- Prove `--doer reference` can pass under WriteScope
- Configure Deep Agents so the judge holds `read_file` only
- Grep for `from loops` and fail if it returns

Spillwave Solutions | spillwave.com

Starting architecture

```
orchestrator  writes nothing, owns budget
planner       steps.jsonl
test_implementer  tests/**
code_implementer  app/**, denied tests/**
judge          read_file only
```

↓

red gate → ten-row rubric → gates.decide

↓

.harness/last-implementer.json

Eight steps in `implementer.run`

1. Read ticket. Refuse if not `ready`.
2. `plan_for`: one test step and one code step per criterion. Derived, not generated. This is Graph Engineering. Not LangGraph.
3. `test_implementer` writes under `tests/**`.
4. Red gate. Empty `new-ids` → escalate.
5. `code_implementer` writes `app/**` until green.
6. Ten-row rubric. No model.
7. `judge_done=None`, so a green rubric is enough. Session 2 still teaches a model judge.
8. `gates.decide`. Trace to `.harness/last-implementer.json`.

Red gate

```
def _new_test_ids(before: set[str], after_failed: set[str]) -> set[str]:  
    return {test_id for test_id in after_failed if test_id not in before}
```

A test that already existed and still fails is not proof of a new contract.

`--doer none` hits this every time. If that run were green, the harness would be lying.

Deep Agents wiring

```
return create_deep_agent(  
    model="anthropic:claude-sonnet-5",  
    tools=[run_tests_tool(repo)],  
    subagents=subagents_for(contract, loop),  
)
```

Judge tools = ["read_file"]. Code-implementer write tool refuses tests/** with a sentence.

create_deep_agent **does not** count retries. Python still owns gates.decide.

Commands

```
cd solutions/sol2_implementer_deep_agents
python3 -m pytest tests -q
python3 harness.py --table-only
python3 harness.py --repo ../../work/northwind-field-crm --ticket T001 --doer none
python3 harness.py --repo ../../work/northwind-field-crm --ticket T001 --doer reference
python3 harness.py --repo ../../work/northwind-field-crm --ticket T001 --doer deep
```

--doer deep **needs** deepagents and a key. Tests do not.

Expected results

--doer none:

```
gate: escalate  
reason: red gate: no new test was observed failing.
```

--doer reference: **copies** `known-good` **into** `tests/**` **then** `app/**`, each phase bound by that role's WriteScope. Ten PASS rows. gate: `pass`.

Spillwave Solutions | spillwave.com

Troubleshooting

SYMPTOM	CAUSE	FIX
<code>task loop:implementer</code> missing	engine deleted	run this <code>harness.py</code>
<code>--doer none</code> is green	red gate not stopping	empty new-ids must escalate
Wrote <code>tests/**</code> as coder	write tool not scoped	sentence refusal
<code>from loops</code> in a file	design leaked	<code>test_standalone.py</code> fails the build

Validation

- `[] pytest:judge tools == ["read_file"]`

- `[] coder write to tests/test_due.py returns REFUSED`

- `[] _new_test_ids({"old"}, {"old","new"}) == {"new"}`

- `[] same signature twice → ESCALATE`

- `[] --table-only prints judge with no repo`

Spillwave Solutions | spillwave.com

Recap

Same role table as Saturday. Enforcement is a missing tool, plus a path check, plus Python on the outside.

If a port imports `loops`, the design leaked.

Spillwave Solutions | spillwave.com

sol2_implementer_agent_sdk

Take-home **configuration port**. It does not run the eight-step loop.

This is the role graph. The live harness is `sol2_implementer_deep_agents`.

It prints the role table and builds `ClaudeAgentOptions`. There is no `implementer.py`, no `gates.py`, and no `rubric.py`. The driver lives in the Deep Agents twin. There is a `tests/` now, and it is what keeps the fence honest.

What it proves

The role table survived the runtime swap.

role	writes	scope	
orchestrator	no	nothing	
planner	yes	steps.jsonl	
test_implementer	yes	tests/**	
code_implementer	yes	app/**, src/**	denied tests/**
judge	no	nothing	

Do not copy this folder into `labs/lab2_implementer/harness.py`. The Saturday stub wants `red_gate / score_attempt / run_loop`. This file exports `cast / build / backend`.

Learning objectives

- Read `roleplan.LOOPS["implementer"]`
- Wire `tools=[...]` plus `PreToolUse`
- Print `--table-only` with no SDK and no key
- Pair this port with `sol2_implementer_deep_agents` when you want the eight steps to run

Spillwave Solutions | spillwave.com

Starting architecture

```
python3 harness.py --table-only
└─ roleplan.plan(contract, "implementer")
    five RolePlans, judge can_write False

python3 harness.py --repo CRM
└─ ClaudeAgentOptions
    agents for planner, test_implementer, code_implementer, judge
    permission_mode=dontAsk
    one PreToolUse scope_hook for the whole cast
```

No call to `implementer.run`.

Scope. Two places, both required

`tools=[...]` decides whether a role can write at all.

`scope_hook` decides which paths. Deny envelope:

```
hookSpecificOutput.hookEventName = PreToolUse
hookSpecificOutput.permissionDecision = deny
```

A typo fails **open**. The field is `maxTurns`, camelCase: `max_turns` raises `TypeError` on the real SDK, and a test fake taking `**kwargs` hides it.

Commands

```
cd solutions/sol2_implementer_agent_sdk
python3 harness.py --table-only
python3 harness.py --repo ../../work/northwind-field-crm
task test    # pytest. task table is --table-only
```

`--table-only` with a missing repo still prints the table and exits 0.

Expected table

Judge must print `no`. Code implementer allow `app/**`, `src/**`, `deny tests/**`.

If judge prints `yes`, the port is wrong. Stop.

Troubleshooting

SYMPTOM	CAUSE	FIX
Thought this ran T001	config port	use Deep Agents harness.py
Judge writes yes	tools leaked Edit	strip judge tools
Fail-open writes	deny shape typo	full hookSpecificOutput
Copied into lab2 stub	wrong artifact	Saturday wants three functions

Validation

- [] table: five roles, judge no
- [] no `implementer.py` in this folder (on purpose)
- [] `task test` needs no key, no SDK, and no clone
- [] you can name the folder that actually runs the loop

Spillwave Solutions | spillwave.com

Recap

Config port, not a filled loop. Same table, different enforcement knob.

The eight steps live in `sol2_implementer_deep_agents`. This folder proves the table survived.

Spillwave Solutions | spillwave.com

Prerequisites

```
cd solutions/sol2_implementer_agent_sdk  
python3 -c "import roleplan, roles, write_scope; print('ok')"
```

No API key. No CRM clone. `claude-agent-sdk` is only needed if you call `options_for` without stubbing.

Saturday Lab 2 still fills `labs/lab2_implementer/harness.py`. This folder is take-home.

Files in this folder

```
SPEC.md Taskfile.yml  
adapter.py contract.py harness.py  
roleplan.py roles.py write_scope.py
```

Missing on purpose: `implementer.py`, `gates.py`, `rubric.py`, `doers.py`, `tests/`.

Same table, three enforcements

RUNTIME

Saturday Claude

Agent SDK (this folder)

Deep Agents

Spillwave Solutions | spillwave.com

HOW JUDGE CANNOT WRITE

YAML tools omit Write

tools omit Edit/Write, plus PreToolUse

subagent tools = [read_file]

Final checklist

- ☐ `--table-only` prints judge `no`
- ☐ `code_implementer` deny includes `tests/**`
- ☐ you did not copy this `harness.py` over the Saturday stub
- ☒ you know `sol2_implementer_deep_agents` is the live loop

Spillwave Solutions | spillwave.com

sol3_research_deep_agents. White paper

A question in, a cited brief out. A topic in, an evidence-backed white paper out.

LangChain Deep Agents. Two entry points. One role table.

PR #156 grew the Saturday brief into a nine-stage report pipeline. Saturday Lab 3 still fills two functions. This folder writes `whitepaper.md`.

Spillwave Solutions | spillwave.com

Two entry points. Same folder.

```
python3 loop.py --question "sqlalchemy nullable datetime column"  
python3 loop.py --paper --topic "context management in multi-agent research loops"
```

The brief is the small version of the paper. Both plan questions, search through one tool boundary, and refuse to ship anything uncited. The paper adds corroboration, figures, and a publish step.

What you will walk through

FILE	HOLDS
loop.py	both entry points, and which cast to print
roleplan.py	research (four roles) and paper (seven)
paper.py	nine stages, three exits
stages.py	each stage's gate
evidence.py	SourceDocument, Claim, Finding
paper_check.py	hard gates on a finished paper
publish.py	secret gist, one id per paper forever
roles.py	three fencing layers

Why this folder exists

LangChain's own Deep Agents quickstart is a research agent. This folder is that shape, pointed at our tool boundary, then grown until it produces something you would hand to a colleague.

A role cannot do a job it was not given a tool for. Nothing about grounding, cost, or stopping is left to a model's own judgment.

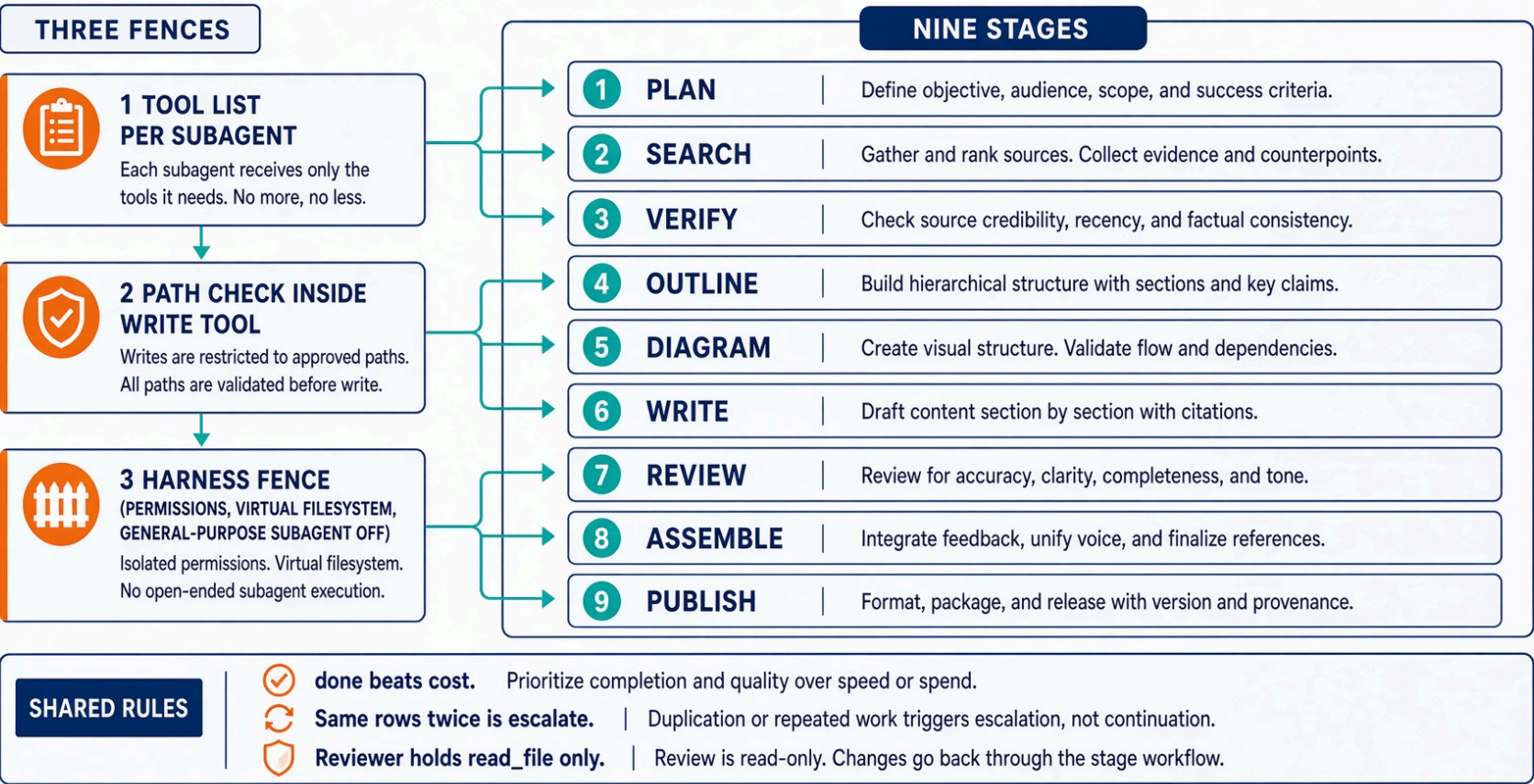
Spillwave Solutions | spillwave.com

Learning objectives

- Print the paper cast: reviewer writes `no`
- Name the three fences Deep Agents actually uses
- Walk nine stages and say which two call no model
- Prefer `done` over a spent budget
- Keep the Saturday brief working (`--question`)
- Refuse `task publish` when `gates.json` is red

Starting architecture

Deep Agents white paper. Nine stages. Three fences.



The paper cast

role	writes	scope
orchestrator	no	nothing
planner	yes	plan.json
researcher	no	nothing
verifier	yes	evidence/** denied: paper/**
diagrammer	yes	diagrams/*.mmd, *.puml
writer	yes	brief.md, paper/**, work/research/**
reviewer	no	nothing

`task table (adds --paper)`. If the reviewer prints `yes`, the translation is wrong and the tests say so.

A role that would hold the same tools as its neighbour is not a role. It is a prompt, and it belongs in a skill.

Three fences. People skip the third.

1. **A tool list per subagent.** The reviewer is handed `read_file` and nothing else. "Do not edit the paper" is not an instruction it can reinterpret.
2. **A path check inside the write tool.** Writer may write `paper/**`. Verifier may write `evidence/**`. Neither can reach the other. Refusal is a sentence, not a traceback: a raw traceback starts a retry loop.
3. **A fence around the harness.** `permissions=`, hide built-in write tools from the orchestrator, `FilesystemBackend(virtual_mode=True)` SO `..` cannot walk out, general-purpose subagent **off**.

Layer 3 is the one people skip. The default general-purpose subagent ships with the harness filesystem tools. Leaving it enabled is how a carefully scoped agent writes anywhere it likes.

Nine stages. Two of them are Python.

#	STAGE	MODEL?	GATE
1	plan	yes	3 to 8 questions, each with a check
2	search	yes	every claim names a source that resolves
3	verify	yes	every important claim has a decided truth state
4	outline	yes	every body section names claim ids that exist
5	diagram	yes	at most 12 nodes, every figure has alt text
6	write	yes	a section cites only the numbers its claims support
7	review	yes	every rubric row passes
8	assemble	no	every hard gate in <code>paper_check</code>
9	publish	no	the gates passed. Opt-in only

Assembling markdown and pushing a gist are things a program does correctly every time. Handing either to a model buys a new failure mode.

Three exits. Done first.

Checked before every stage:

EXIT	WHEN
done	stage 8 finished and every hard gate is green
cost	the money budget is spent
max turns	a stage exhausted its retries

Done beats a spent budget. A run that finished and then noticed it was over its cap did finish. Reporting that as a cost failure discards the paper.

Inside a stage, `gates.decide` short-circuits the same rows failing twice.

One boundary. Four backends.

The loop never learns which one answered.

BACKEND	WHEN
perplexity	PERPLEXITY_API_KEY. MCP first, then REST
context7	ctx7 on PATH, or the MCP server. Verification only
websearch	coding agent through an inbox file
fixture	always. A recording, for a room with no network

task paper is fixture. task live is auto.

Saturday brief is still here

```
def plan_questions(question: str) -> list[str]:  
    return [question, f"{question} common mistake", f"{question} how to verify"]  
  
def check_brief(body, sources):  
    return brief.check(body, sources)
```

Four arithmetic rows: `has_sources`, `grounded`, `cited`, `style`.

```
task brief -- --question "sqlalchemy nullable datetime column"
```

That is the Lab 3 answer. The paper is the take-home grown from it.

Evidence records outlive the paper

```
work/paper/<slug>/
  whitepaper.md
  plan.json
  outline.json
  evidence/      SourceDocument, Claim, Finding
  diagrams/      mermaid and plantuml sources
  figures/       SVG, polished PNG, sidecar
  gates.json
  .paper-state.json  resume reads this
```

Same record shape as `loop_eng_2nd_brain/knowledge/research/`. A run can be ingested later without a conversion step. The claims outlive the paper, which is the point of writing them down separately.

Hard gates on the finished paper

`paper_check.py`. No model. A failing hard gate blocks publish.

Required sections: abstract, introduction, references. Recommended: limitations.

Also: min 400 words, figures need alt text, diagram source must not survive into the body, citation arithmetic reused from `brief.py`.

`signature()` is the sorted names of **hard** failures. A soft warning that keeps firing would look like a stall and escalate a run that is converging.

Publish. Four load-bearing rules.

1. **One gist per paper, forever.** Id in `gist-ids.tsv`. Refresh, do not mint a new gist per run.
2. **Secret, never public.** Unlisted, not private. The URL is the credential.
3. **Figures inline.** A gist is flat. Copy figures to the root, rewrite image links to raw URLs.
4. **The gate comes first.** `push` refuses when `gates.json` is red.

`task paper` never publishes. `task publish` does.

```
task publish -- --slug ... --dry-run --out /tmp/staged
```

Resume and fixtures

A resumed live run skips every finished stage and carries the cost forward.

The fixture runner keys replies by a phrase in the prompt, not by position.

A positional queue restarts at zero when the run resumes at stage six and hands the writer the outline reply.

A list of replies repeats the last entry. That repeat is deliberate: a retry gets the same answer, which is the stable failure `gates.decide` exists to catch, so the offline run can demonstrate escalate without anybody faking it.

Commands

```
cd solutions/sol3_research_deep_agents
python3 -m venv .venv && source .venv/bin/activate && pip install pytest
task table          # paper cast, no SDK
task checks         # evidence, paper_check, diagrams, publish, mcp_tools
task test           # 198 tests. No SDK, no key, no network
task paper          # nine stages, fixture backend
task setup          # only for a live run
task live TOPIC="context management in multi-agent research loops"
task publish -- --slug context-management-in-multi-agent-research-loops
```

Standalone. Tests pin it.

tests/test_standalone.py fails if this folder imports a shared engine.

Other pins:

TEST	ASSERTS
role table	reviewer holds no write
stages	plan 3 to 8 questions; outline binds real claim ids
paper_check	missing abstract fails; source syntax in body fails
publish	red gates.json refuses
research	[9] is ungrounded; budget hard cap raises

Troubleshooting

SYMPTOM	CAUSE	FIX
Reviewer writes <code>yes</code>	tool list leaked	<code>read_file</code> only
Agent writes anywhere	general-purpose subagent left on	turn it off
Traceback retry storm	write tool raised	return a refusal sentence
Resume hands wrong reply	positional fixture	key by prompt phrase
New gist every run	ignored <code>gist-ids.tsv</code>	one id per slug
Published a red paper	skipped <code>gates.json</code>	<code>push</code> must refuse
Saturday brief missing	ran <code>--paper</code> by accident	<code>--question</code> is the lab

Validation checklist

- ☐ `task table:reviewer` no
- ☐ `task test` is 198, no SDK
- ☐ `task paper` **writes** `whitepaper.md` **plus** `evidence/`
- ☐ Stage 8 and 9 call no model
- ☐ Done beats cost
- ☐ `task publish -- --dry-run` restages figures
- ☐ `tests/test_standalone.py` **still passes**

Spillwave Solutions | spillwave.com

Recap

What we built. A brief for Saturday. A paper for take-home. Same exits.

Takeaways

1. Two entry points. One table.
2. Three fences. The third is the one people skip.
3. Assemble and publish are Python.
4. Done beats cost.
5. Claims outlive the paper.

Closing line. A role that holds the same tools as its neighbour is a prompt.

sol3_research_agent_sdk. White paper

A topic in. An evidence-backed technical white paper out.

Claude Agent SDK. Seven roles. Python owns the phases.

This is **not** the old config-only port. PRs #157 / #158 grew it into a report generator. Saturday Lab 3 still fills two functions. This folder writes `paper.md`.

Spillwave Solutions | spillwave.com

What you will build (already filled)

ARTIFACT	PATH
the paper	work/<slug>/paper.md
figures	work/<slug>/diagrams/*.png
knowledge bundle	work/<slug>/knowledge/research/
checkpoint	work/<slug>/.harness/state.json

Cast lives in `roleplan.py`. Pipeline lives in `paper.py`. One PreToolUse hook in `roles.py`. Checks in `checks.py`. Exits in `gates.py`.

Why this folder exists

The same loop, in a different runtime. The rubric, the red gate, the write scope, and the exits did not have to change to make it run.

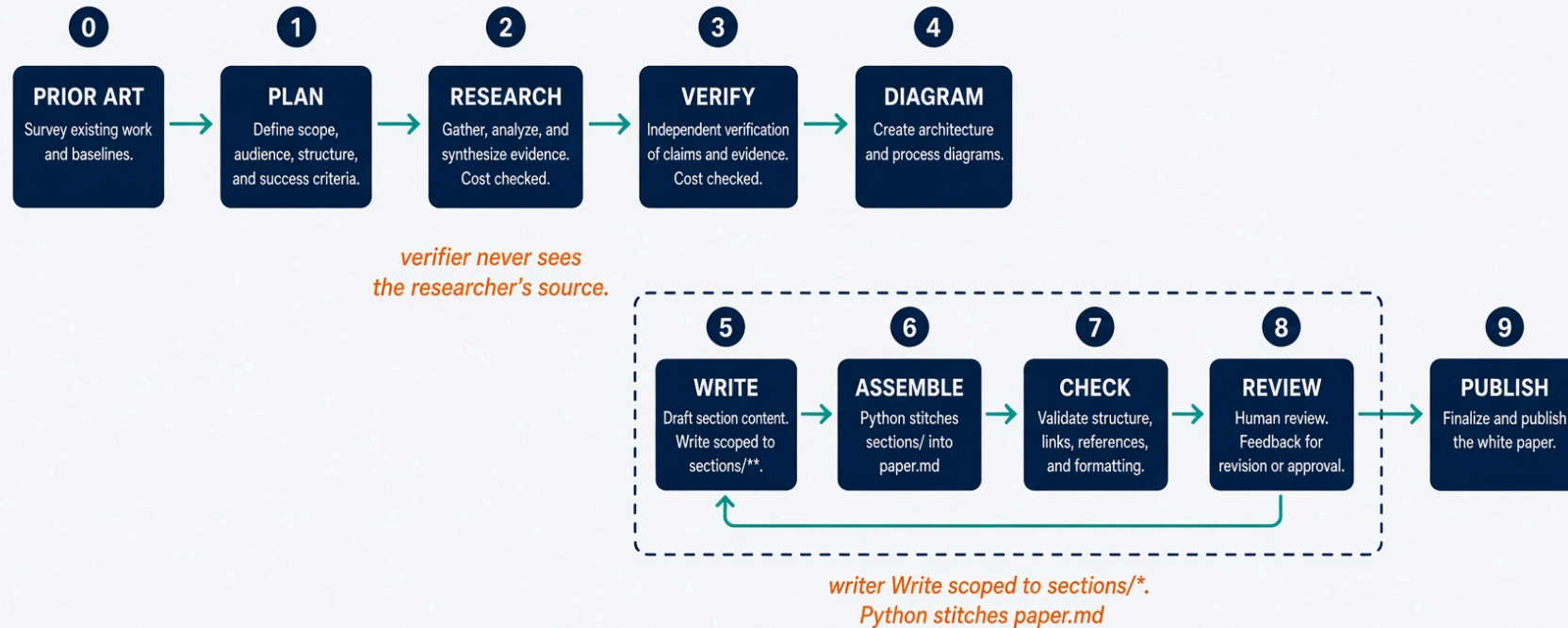
A searcher that can write can edit the evidence to fit the paper. A verifier shown the researcher's answer is not a second opinion. A model handed Bash widens the blast radius from "a wrong paper" to "anything this machine can run".

Learning objectives

- Print the seven-role table with no SDK installed
- Explain why `allowed_tools` is a session-wide allowlist
- Declare MCP in Python, not via a gitignored `.mcp.json`
- Cap questions, claims, and dollars as three separate budgets
- Check cost **inside** research and verify, not only at the gate
- Refuse to publish a paper that did not pass

Starting architecture

Agent SDK white paper. Ten phases. Python owns the order.



Three exits: pass / retry / escalate. Cost checked inside research and verify, not only at the gate.

The cast. Seven roles, one writer.

orchestrator	Task	writes nothing
planner	Read, Glob, Grep	returns plan JSON. Python writes plan.json
researcher	Read + search MCP	writes nothing
verifier	Read + search MCP	writes nothing. Never sees the source.
diagrammer	Read, Glob, Grep	returns diagram source. Python renders.
writer	Read + Write	sections/** only. Cannot reach paper.md
judge	Read, Glob, Grep	writes nothing

task table. Judge, researcher, verifier must print `no` in the writes column.
If any prints `yes`, stop.

Two places enforce scope. You need both.

```
tools=[...]      decides whether a role can write at all
PreToolUse hook  decides which paths it may write
```

One hook serves the whole cast. It reads `agent_type` off the tool call.

`sol` registered one hook per writing role. That does not generalize: several hooks on `write` all run, an empty dict means "no opinion", and the first role that shrugs lets another role's write through.

A write with no `agent_type` is denied. So is a write from anyone but the writer. So is a write to `paper.md`, `claims.json`, or outside the work dir.

`allowed_tools` is not the parent's list

It is a session-wide permission allowlist. It gates subagents too.

First live run: options said `allowed_tools=["Agent"]` with `permission_mode="dontAsk"`. Researcher held Perplexity, Context7, WebSearch. Every call came back denied.

What the researcher then did is the part worth keeping. It did **not** answer from memory. It reported "NO RESEARCH WAS PERFORMED", named each denied tool, and returned an empty claim list. The run escalated with "no source produced a single claim" rather than shipping four fabricated sections.

`options_for` now allows the union of what the cast holds. `Bash` is denied at the top. The parent is kept from writing by the hook, not the allowlist.

MCP is declared in Python

```
def mcp_servers() -> dict:
    servers = {"context7": {"type": "http", "url": "https://mcp.context7.com/mcp"}}
    key = os.environ.get("PERPLEXITY_API_KEY")
    if key:
        servers["perplexity-ask"] = {...}
    return servers
```

`strict_mcp_config=True`. `.mcp.json` holds a key, so it is gitignored. A fresh clone has no such file. The researcher then loses both servers with **no error**, and a topic it cannot search reads as a topic nobody has written about.

Perplexity is included only when the key is set. An empty key is an auth error on every question, which is the same failure wearing a different hat.

Ten phases. Python owns the order.

0 prior_art	second brain, skip if missing	-> prior-art.md
1 plan	sections and questions	-> plan.json
2 research	claims from primary sources	-> sources.json, claims.json
3 verify	independent second look	-> verdicts.json
4 diagram	source in, Python renders	-> diagrams.json
5 write	one section at a time	-> sections/*.md
6 assemble	stitch + references, in Python	-> paper.md
7 check	seven deterministic rows	-> check.json
8 review	judge on what a script cannot	-> review.json
9 publish	private gist, on request	-> gist.json

Phases 0 to 4 run once. 5 to 8 are the retry cycle. Re-running research because a paragraph lost its citation marker buys a bill, not a better paper.

The planner is told the budget

```
--max-questions default 12. --max-diagrams default 4.
```

A live planner returned seven sections and twenty-eight questions. Truncating to four questions left a paper with two sections and five orphaned headings.

A planner that knows the ceiling writes a whole paper under it. Python still enforces the cap afterwards.

Verify is independent. Claims are capped.

The verifier is given the claim text and nothing else. A claim the two disagree on is `disputed`. One the verifier cannot reach is `unverified`, never silently dropped.

Four questions produced 111 claims on one live run. Every claim is a verification turn.

`--max-claims` default 24. Claims with a number, a version, or a date are checked first. The rest stay `unverified`. The writer states them qualitatively. The knowledge bundle records that nobody checked them.

Cost is checked inside the phases

`--max-usd` default 5.00. Checked inside research, inside verification, and at the gate.

Checking only at the gate is a bug this port had: the gate runs once per attempt, and a twenty-four question research phase can spend the whole budget several times over before the gate ever sees it.

Running out mid-verification leaves remaining claims `unverified`. Marking them verified because the money ran out is the lie.

Seven deterministic rows. No model vote.

ROW	PASS WHEN
complete	every section the plan named is in the paper
grounded	every [n] resolves to a retrieved source
cited	every claim paragraph names a source
sourced	every arXiv / DOI / author-year appears in retrieved text
images	every figure the paper references is a file on disk
style	zero em dashes outside code spans
(plus sources)	the reference list is intact

complete looks redundant. A run that spent its budget in verification wrote no sections at all: title, abstract, 25-entry reference list. Every other row passed. The judge caught it. The row is there so the first backstop does not have to.

The check that matters most: `sourced`

A web search cannot refute a citation that was never published. Asking a model whether a reference is real gets you a confident yes.

The only thing that catches a fabricated arXiv id or DOI is looking for it in the text that was actually retrieved. `checks.ungrounded_identifiers`.

Three exits. Stall is the one people miss.

`gates.py`: pass, retry, escalate. No fourth.

When an attempt fails in exactly the same way as the last one, the loop is not converging. The signature is **what** failed, not how it was worded. A model rephrasing its own complaint does not read as progress.

A paper that did not pass is never published.

Spillwave Solutions | spillwave.com

Commands. Offline first.

```
cd solutions/sol3_research_agent_sdk
task table          # no SDK, no key, no network
task checks         # checks / gates / publish / rkc --demo
task test           # unit tests, same constraints
task setup          # SDK + imagen-diagrams clone into .cache/
task demo           # full offline run over the fixture
task run TOPIC="how MCP servers authenticate"
task publish TOPIC="..." # also pushes a private gist
```

Expected output

```
work/<slug>/  
  paper.md  
  diagrams/*.png  
  knowledge/research/      RKC: sources, claims, evidence, findings  
  .harness/state.json      per-phase status and cost
```

Deleting one phase output re-runs that phase and no other. Resume needs no bookkeeping: a phase whose file exists already ran.

`--private` on a gist means unlisted, not access controlled. Anyone holding the URL can read the paper and fetch every figure. Treat the URL as the credential.

Plugin files vs Python

```
plugin/agents/research-{{planner, researcher, verifier, diagrammer, writer, judge}}.md  
plugin/skills/research-loop/SKILL.md
```

The skill is the readable specification of what `paper.py` implements. Do **not** run the skill from this port. Python owns the phases. Running the skill would give you two orchestrators disagreeing about whose turn it is.

When an agent markdown file and the role table disagree, `options_for` raises.

Troubleshooting

SYMPTOM	CAUSE	FIX
Subagent tools denied	<code>allowed_tools</code> too narrow	union of the cast, plus hook
Silent empty research	inherited missing <code>.mcp.json</code>	<code>mcp_servers()</code> in Python
Fabricated sections	researcher answered from memory	empty claims must escalate
Bill before a gate	cost only at the gate	check inside research and verify
Title plus references, no body	spent budget in verify	<code>complete</code> row, then judge
Writer rewrote <code>paper.md</code>	hook not looking at <code>agent_type</code>	deny anyone but writer, deny that path

Validation checklist

- ☐ `task table`: one writer, everyone else `no`
- ☐ `task test` needs no SDK, no key, no network
- ☐ Verifier is never handed the researcher's source
- ☐ Writer cannot reach `paper.md` or `claims.json`
- ☐ A write with no `agent_type` is denied
- ☐ `maxTurns` not `max_turns` (TypeError on the real SDK)
- ☐ A paper that did not pass is never published

Spillwave Solutions | spillwave.com

Recap

What we built. A topic in, a paper plus a knowledge bundle out.

Takeaways

1. Trigger and runtime change. Exits do not.
2. `allowed_tools` gates the children.
3. MCP in Python, or a fresh clone silently cannot search.
4. Three budgets, and cost inside the phases.
5. `sourced` catches a citation a model will bless.

Closing line. The grounding contract has to hold while the wiring is wrong.

sol4_fixer_agent_sdk

The working Module 4 loop. Issue 120.

`query(), not ClaudeSDKClient.permission_mode: dontAsk.` Merge is never a tool.

Spillwave Solutions | spillwave.com

Layout

loop.py	summarize_failure + repair_until_green wrappers
fixer.py	the while loop
doers.py	none / reference / cli / SDK backend
gates.py	identical copy to sol3
contract.py	Taskfile + junit + coverage
roles.py	dontAsk, one PreToolUse hook, deny tests/**
tests/	hook deny/allow, same-signature escalate

No `unattended.py`. Durable `.harness/state.json` and `exit 0/2/1` lived there and were not copied. This CLI returns `0` if green else `1`. Name that drift.

Learning objectives

- Walk `fixer.run`
- Set `permission_mode: dontAsk` because nobody is in the chair
- Deny `tests/**` on the code implementer
- Stash before `--branch broken-pr`
- Leave a comment when the loop gives up

Starting architecture

```
Trigger → fixer.run
        contract.run("test")
        green? PASS
        never ran? ESCALATE
        same failing ids twice? ESCALATE + comment
        else doer repairs inside app/**
→ .harness/last-fixer.json
→ Human merge
```

Five unattended lines

```
permission_mode: dontAsk      # never prompts, and fails closed
PreToolUse deny tests/**     # cannot weaken a test
maxTurns=12                   # camelCase. max_turns= is a TypeError
tests after every turn = pytest # not a claim
Merge is never a tool
```

This port used `acceptEdits` and that was the bug. Both modes never prompt. `acceptEdits` auto-accepts every edit **before** the allow list is read, so the hook is the only fence, and the hook fails open on a typo. `dontAsk` denies anything not pre-approved, which makes `allowed_tools` a real second gate.

"Nobody is in the chair" is the argument for `dontAsk`, not against it.

failure_summary

```
def failure_summary(run_result) -> str:
    failed = sorted(run_result.junit.failed_ids)
    lines = [f"{len(failed)} failing: {'', '}.join(failed[:5])}" if failed else ["the suite is red"]
    error = ERROR_IN_OUTPUT.search(run_result.output or "")
    if error:
        lines.append(error.group(0).strip()[:200])
    return "\n".join(lines)
```

Doers

	SPEC	BEHAVIOR
none		writes nothing. Loop still reports the truth
reference		copies <code>known-good</code> inside <code>WriteScope</code>
sdk		<code>AgentSdkBackend</code> wrapping <code>query()</code>
<code>claude / codex / grok</code>		<code>CliBackend</code>

Reference skipping `tests/**` is the whole point.

Commands

```
cd solutions/sol4_fixer_agent_sdk
python3 -m pytest tests -q
python3 loop.py --table-only
# after stash:
python3 loop.py --repo ../../work/northwind-field-crm --branch broken-pr --doer none
python3 loop.py --repo ../../work/northwind-field-crm --branch broken-pr --doer reference
```

Expected `--doer none`

```
attempt 1: 1 failing -> retry
attempt 2: 1 failing -> escalate
gate: escalate
The fixer gave up.
A human should take this one.
```

Tests

TEST

test_hook_denies_test_write

test_hook_allows_app_write

test_the_permission_mode_denies_rather_than_prompts

test_same_failing_ids_escalate

test_no_loops_import

Spillwave Solutions | spillwave.com

ASSERTS

Write tests/test_x.py → deny

Write app/main.py → {}

permission_mode == "dontAsk"

same signature → ESCALATE

no from loops

Troubleshooting

SYMPTOM	CAUSE	FIX
Dirty tree	Lab 2 leftover	stash
Green on <code>none</code>	missing signature stop	<code>gates.decide</code>
Edited a test	hook fail-open	full deny envelope
Exit 1 on escalate	unattended.py missing	named drift, not a silent bug

Recap

Same graph, nobody at the keyboard. Reference doer still bound by WriteScope. Human owns merge.

Rebuild `unattended.py` in this folder if you need exit 2 for escalate.

Spillwave Solutions | spillwave.com

Prerequisites

```
cd solutions/sol4_fixer_agent_sdk  
python3 -m pytest tests -q  
git -C ../../work/northwind-field-crm stash --include-untracked
```

--doer sdk **needs** claude-agent-sdk. --doer none **and** --doer reference **do not**.

`repair_until_green` **wrapper**

```
def repair_until_green(contract, budget: int = 3, doer: str = "reference") -> dict:
    if doer == "sdk":
        doer = backend(contract)
    return fixer.run(repo=contract.repo, budget=budget, doer=doer, research_backend=None)
```

Saturday types a body. This folder delegates to `fixer.run`.

Final checklist

- [] pytest green without a key
- [] table: judge no, coder denied tests/**
- [] permission_mode == "dontAsk"
- [] --doer none on broken-pr escalates with a comment
- [] merge is not in any tool list

Spillwave Solutions | spillwave.com

sol4_fixer_deep_agents

Configuration port. It does **not** run the fixer.

This is the role graph. The live harness is `sol4_fixer_agent_sdk`.

No `fixer.py`, `gates.py`, `doers.py`, **or** `tests.task test` **is** `--table-only`.

What it proves

role	writes	scope	
orchestrator	no	nothing	
code_implementer	yes	app/**, src/**	denied tests/**
judge	no	nothing	

Live repair is `sol4_fixer_agent_sdk.backend(contract)` exists. `main` never calls it.

Learning objectives

- Read `LOOPS["fixer"]` (three roles, no planner)
- Hand the coder a scoped write tool
- Hand the judge `read_file` only
- Refuse `tests/**` with a sentence
- Name the folder that actually repairs `broken-pr`

Starting architecture

```
python loop.py --table-only
└─ three RolePlans

python loop.py --repo CRM
└─ create_deep_agent subagents
    code-implementer: read_file + write_code_implementer
    judge: read_file
```

No while loop. No junit. No comment.

Spillwave Solutions | spillwave.com

Scoped write tool

```
@tool(f"write_{role.name}")
def write(path: str, content: str) -> str:
    try:
        scope.check(path)
    except ScopeViolation:
        return f"REFUSED. {role.name} may write {allowed}. {path} is outside that scope."
```

Deny tests/**. **Allow** app/**, src/**. Refusal is a string, not an exception.

Commands

```
cd solutions/sol4_fixer_deep_agents
python loop.py --table-only --repo ../../work/northwind-field-crm
python loop.py --repo ../../work/northwind-field-crm
task test
```

Missing repo plus `--table-only`: prints declared scopes, exit 0. Without `--table-only`, `ContractError` raises.

Expected table

Judge prints `no`. Code implementer prints `yes` with `deny tests/**`.

If judge prints `yes`, stop.

Spillwave Solutions | spillwave.com

Troubleshooting

SYMPTOM	CAUSE	FIX
Expected a green branch	config port	Agent SDK folder
Judge writes yes	write tool handed to judge	tools = [read_file]
Raises on missing repo	forgot --table-only	table-only is the self-check

Validation

- [] table: three roles
- [] judge no
- [] coder denied tests/**
- [] you can name sol4_fixer_agent_sdk as the live loop

Spillwave Solutions | spillwave.com

Recap

Same table, Deep Agents enforcement knob. This is graph without a loop. Use the Agent SDK folder when you want a green branch or an honest comment.

Spillwave Solutions | spillwave.com

Prerequisites

```
cd solutions/sol4_fixer_deep_agents  
python loop.py --table-only
```

DEFAULT_LOOP in this copy is "implementer". So is sol1's. Both inherited it from the shared `roleplan.py` this repo deleted, and neither changed it. `loop.py` sets `LOOP = "fixer"` and passes it at every call site. Without that you get five roles instead of three.

The rule is that a caller names its loop. A test in each folder pins it, because the default is one edit away from being wrong everywhere.

Files in this folder

```
SPEC.md Taskfile.yml  
adapter.py contract.py loop.py  
roleplan.py roles.py write_scope.py
```

Missing on purpose: `fixer.py`, `doers.py`, `gates.py`, `tests/`.

Why three roles, not five

The work is already defined by what is red. No planner. No test implementer. The judge reads junit. The coder may write `app/**` and may not write `tests/**`.

Spillwave Solutions | spillwave.com

Final checklist

- `[] LOOP = "fixer"`
- `[] table: three rows, judge no`
- `[] coder write tool refuses tests/**`
- `[] live repair is sol4_fixer_agent_sdk`

Spillwave Solutions | spillwave.com